

Table of Contents

Table of Contents	1
1. JDK vs JRE vs JVM — How Java Code Runs	3
1.1 Simple Intuition	3
1.2 Container Hierarchy	3
1.3 How Java Code Runs — End to End	3
Stage 1 — Compilation (javac)	3
Stage 2 — Class Loading	4
Stage 3 — JVM Memory (Runtime Data Areas)	4
Stage 4 — Execution Engine (Interpreter + JIT)	4
1.4 Common Traps	5
2. Primitive Data Types, Autoboxing & Typecasting	6
2.1 The 8 Primitive Types	6
2.2 Arithmetic Traps	6
2.3 Autoboxing and Unboxing	7
The Integer Cache	7
Performance Trap — Autoboxing in Loops	7
NPE on Unboxing	8
2.4 Typecasting	8
Primitive Casting — Widening vs Narrowing	8
Object Casting — Reference Types	8
Numeric Promotion	9
2.5 Interview Traps	9
3. Arrays and Varargs	11
3.1 Array Internals	11
3.2 Multi-dimensional Arrays	11
3.3 Key Array Operations	12
3.4 Varargs — Compiler Desugaring	12
Varargs and Generics — Heap Pollution	12
3.5 Common Traps	13
4. Class vs Object	14
4.1 Core Distinction	14
4.2 Object Creation — JVM Steps	14

4.3 The Object Header	15
4.4 Code Example	15
4.5 this — The Hidden Parameter	16
4.6 null — A Reference to No Object	17
4.7 Interview Traps	17

1. JDK vs JRE vs JVM — How Java Code Runs

1.1 Simple Intuition

Java has three concentric rings of responsibility. The **JVM** (Java Virtual Machine) is the engine — it runs `.class` files (bytecode). The **JRE** (Java Runtime Environment) is the engine plus standard libraries (`java.util`, `java.io`, etc.). The **JDK** (Java Development Kit) adds developer tools: `javac`, `jdb`, `jmap`, `jstack`.

Key insight: The JVM is the reason Java is platform-independent. Compile once to bytecode (.class), and the JVM for each platform translates it to native machine code at runtime.

1.2 Container Hierarchy

Layer	Contains
JDK	JRE + <code>javac</code> , <code>javadoc</code> , <code>jar</code> , <code>jdb</code> , <code>jconsole</code> , <code>jmap</code> , <code>jstack</code> , <code>jstat</code>
JRE	JVM + standard class libraries (<code>java.lang</code> , <code>java.util</code> , <code>java.io</code> , <code>java.net</code> , <code>java.sql</code> , <code>java.nio</code>)
JVM	Class loader, Bytecode verifier, Runtime data areas, Execution engine (Interpreter + JIT + GC)

1.3 How Java Code Runs — End to End

`YourCode.java` → `javac` → `YourCode.class` (bytecode) → JVM → Native machine code

Stage 1 — Compilation (`javac`)

`javac` is **not** a traditional compiler — it produces platform-neutral bytecode, not native machine code. Internally it: lexes source into tokens, parses into an AST, performs semantic analysis, then emits a `.class` file. Every `.class` file begins with the magic number `CA FE BA BE`.

```
// Simple bytecode for System.out.println("Hi")

0: getstatic #7 // Field java/lang/System.out

3: ldc #13 // String "Hi"

5: invokevirtual #15 // Method println:(Ljava/lang/String;)V

8: return
```

Stage 2 — Class Loading

Classes are loaded **lazily** — on first use. Three built-in ClassLoaders form a delegation hierarchy using the parent-delegation model.

```
Bootstrap ClassLoader ← loads java.base (written in C++)  
  
↑  
  
Platform ClassLoader ← loads java.sql, java.xml, etc.  
  
↑  
  
Application ClassLoader ← loads your application classpath
```

Phase	What happens
Loading	Reads .class bytes, creates Class object in Metaspace
Linking	Verifies bytecode safety, allocates static memory, resolves symbolic refs
Initialization	Executes — static initializers and static blocks

Stage 3 — JVM Memory (Runtime Data Areas)

Area	Scope	Stores	OOM Error
Heap — Eden	Shared	New object allocations	OutOfMemoryError
Heap — Old/Tenured	Shared	Long-lived objects (promoted after ~15 GC cycles)	OutOfMemoryError
Metaspace	Shared	Class metadata, method bytecode, static refs	OutOfMemoryError
Code Cache	Shared	JIT-compiled native code	JIT stops
JVM Stack	Per-thread	Stack frames: locals, operand stack, CP ref	StackOverflowError
PC Register	Per-thread	Address of current bytecode instruction	—

Metaspace replaced PermGen in Java 8. It lives in native memory and grows until native memory is exhausted. Always set `-XX:MaxMetaspaceSize` in production to prevent classloader leak OOMs.

Stage 4 — Execution Engine (Interpreter + JIT)

The JVM uses a two-mode strategy. Initially it interprets bytecode. After a method becomes "hot" (called N times), the JIT kicks in.

Compiler	Alias	Trigger	What it does
C1	Client compiler	Early / low invocation count	Fast compile, light optimisations

C2	Server compiler	After rich profiling data	Aggressive: inlining, escape analysis, loop unrolling
----	-----------------	---------------------------	---

Key JIT optimisations: **Inlining** (replaces call with method body), **Escape analysis** (stack-allocates objects that don't escape), **Dead code elimination**, **Loop unrolling**.

1.4 Common Traps

Trap: "JVM is platform independent" — Wrong. The BYTECODE is platform independent. The JVM itself is a native binary compiled per OS/architecture.

Trap: Metaspace can't OOM — It can. Classloader leaks (common in redeployed webapps) grow Metaspace until native memory is exhausted.

Trap: "Objects always live on the heap" — Not true. Escape analysis allows short-lived objects to be stack-allocated by JIT.

Trap: "Bytecode is interpreted line by line" — Only initially. JIT compiles hot paths to native code. After warmup, Java runs at near-native speed.

2. Primitive Data Types, Autoboxing & Typecasting

2.1 The 8 Primitive Types

Primitives are raw binary values stored directly on the stack (or inline in arrays/objects on the heap). No object header, no reference, no GC overhead — just bits.

Type	Size	Range / Values	Default	Wrapper
byte	8 bits	-128 to 127	0	Byte
short	16 bits	-32,768 to 32,767	0	Short
int	32 bits	-2^{31} to $2^{31}-1$ (~2.1B)	0	Integer
long	64 bits	-2^{63} to $2^{63}-1$	0L	Long
float	32 bits	~7 decimal digits precision (IEEE 754)	0.0f	Float
double	64 bits	~15 decimal digits precision (IEEE 754)	0.0d	Double
char	16 bits	Unicode U+0000 to U+FFFF	'\u0000'	Character
boolean	1 bit*	true / false (*JVM maps to int internally)	false	Boolean

boolean: The JVM spec does not define its exact bit size. A single boolean is typically mapped to a 4-byte int for alignment. In a boolean[] each element is stored as a byte.

2.2 Arithmetic Traps

```
int a = Integer.MAX_VALUE; // 2_147_483_647

int b = a + 1; // -2_147_483_648 – silent overflow, no exception!

// Fix: use long

long safe = (long) a + 1; // 2_147_483_648

// Or throw on overflow:

int checked = Math.addExact(a, 1); // ArithmeticException

// Floating-point is NOT precise

double d = 0.1 + 0.2;

System.out.println(d); // 0.30000000000000004 – never use for money
```

```
// For money: always BigDecimal with String constructor

BigDecimal price = new BigDecimal("0.10").add(new BigDecimal("0.20"));
```

2.3 Autoboxing and Unboxing

Autoboxing is a **compiler transformation**, not a JVM feature. The compiler rewrites `Integer x = 42` into `Integer x = Integer.valueOf(42)`. Unboxing rewrites `int y = x` into `int y = x.intValue()`.

The Integer Cache

`Integer.valueOf()` uses a cache for values **-128 to 127** (configurable via `-XX:AutoBoxCacheMax=N`). Outside this range, a new heap object is created every time.

```
Integer a = 100;

Integer b = 100;

System.out.println(a == b); // true – same cached object

Integer c = 200;

Integer d = 200;

System.out.println(c == d); // false – different heap objects!

System.out.println(c.equals(d)); // true – always use .equals()
```

Always use `.equals()` for Integer comparison. `==` works by accident for values in `[-128, 127]` but silently fails outside that range. Cache applies to: Byte, Short, Long `(-128..127)`, Character `(0..127)`, Boolean (TRUE/FALSE). Float and Double have NO cache.

Performance Trap — Autoboxing in Loops

```
// CATASTROPHIC – creates 10 million Long objects

Long sum = 0L;

for (long i = 0; i < 10_000_000; i++) {

    sum += i; // unbox → add → rebox → new Long every iteration

}

// CORRECT – pure primitive arithmetic

long sum = 0L;
```

```
for (long i = 0; i < 10_000_000; i++) {  
  
    sum += i; // no allocation, no GC  
  
}
```

NPE on Unboxing

```
Map<String, Integer> counts = new HashMap<>();  
  
int count = counts.get("missing"); // NPE! get() returns null, unboxing explodes  
  
// Fix:  
  
int count = countsOrDefault("missing", 0);  
  
// Also:  
  
Integer nullable = null;  
  
int val = nullable; // NullPointerException – unboxing null
```

2.4 Typecasting

Primitive Casting — Widening vs Narrowing

```
// Widening – implicit, no data loss  
  
int i = 100;  
  
long l = i; // implicit widening  
  
double d = i; // implicit widening  
  
// Narrowing – explicit cast required, data loss possible  
  
double pi = 3.14159;  
  
int truncated = (int) pi; // 3 – fractional part TRUNCATED (not rounded)  
  
long val = 300L;  
  
byte b = (byte) val; // 44 – 300 % 256 = 44, wraps around
```

Object Casting — Reference Types

```
// Upcasting – always safe, implicit
```

```

Animal animal = new Dog(); // Dog IS-A Animal

// Downcasting – unsafe, explicit, can throw ClassCastException

Dog dog = (Dog) animal; // JVM: checkcast bytecode verifies actual type

// Safe pattern – Java 16+ pattern matching instanceof

if (animal instanceof Dog d) {

    d.bark(); // no explicit cast needed

}

// Wrong: no hierarchy between Integer and Long

Object obj = Integer.valueOf(42);

Long l = (Long) obj; // ClassCastException at runtime!

long l2 = ((Integer) obj).longValue(); // correct

```

Numeric Promotion

Before arithmetic on types smaller than `int`, the JVM promotes them to `int`.

```

byte a = 10, b = 20;

byte c = a + b; // COMPILER ERROR: a+b is int, cannot assign to byte

byte c = (byte)(a + b); // explicit narrowing required

char ch = 'A'; // Unicode 65

int result = ch + 1; // 66 – char promoted to int

char next = (char)(ch + 1); // 'B' – must cast back

```

2.5 Interview Traps

`==` on Integer: works for `[-128, 127]`, silently fails outside. Always `.equals()`.

float/double `==` comparison: `0.1 + 0.2 != 0.3` due to IEEE 754 binary imprecision. Use `Math.abs(a - b) < epsilon` or `BigDecimal` for money.

`char + char` is int: `'a' + 'b' = 195`, not `"ab"`. Use String concatenation or `StringBuilder`.

Array covariance: `String[]` is assignable to `Object[]` but writing a non-String element throws `ArrayStoreException` at runtime.

3.3 Key Array Operations

Operation	Method	Algorithm / Notes
Sort primitives	<code>Arrays.sort(int[])</code>	Dual-Pivot Quicksort — $O(n \log n)$ average, in-place, unstable
Sort objects	<code>Arrays.sort(Object[])</code>	TimSort — $O(n \log n)$, stable, requires Comparable/Comparator
Copy	<code>Arrays.copyOf()</code> / <code>System.arraycopy()</code>	<code>System.arraycopy</code> is native (memcpy-equivalent) — fastest option
Slice	<code>Arrays.copyOfRange(src, from, to)</code>	Indices [from, to) — allocates new array
Fill	<code>Arrays.fill(arr, val)</code>	Fills all elements; use per-row for multi-dim arrays
Compare	<code>Arrays.equals()</code> / <code>Arrays.deepEquals()</code>	<code>a.equals(b)</code> is reference check — always use <code>Arrays.equals()</code>
Search	<code>Arrays.binarySearch(sorted, key)</code>	Array MUST be sorted first — returns negative index if absent

3.4 Varargs — Compiler Desugaring

`Varargs (...)` is a **pure compiler feature**. The method signature `void log(String... msgs)` compiles to `void log(String[] msgs)`. Every call site that uses varargs syntax causes the compiler to emit a new `String[]{...}` allocation.

```
// What you write:

log("INFO", "started", "connected");

// What the compiler generates:

log("INFO", new String[]{"started", "connected"});

// Zero-arg call – empty array allocated, not null:

log("DEBUG"); → log("DEBUG", new String[0]);

// Passing an array directly – NO extra allocation:

String[] msgs = {"a", "b"};

log("INFO", msgs); // array passed as-is, no new array created
```

Varargs and Generics — Heap Pollution

```
// UNSAFE – compiler warns "unchecked"

public static <T> List<T> listOf(T... elements) { ... }
```

```
// Runtime type of elements is Object[], not T[]

// This is heap pollution – variable of type T[] refers to Object[]

// SAFE – mark with @SafeVarargs when:

// 1. You don't write into the varargs array

// 2. You don't expose the varargs array reference

@SafeVarargs

public static <T> List<T> listOf(T... elements) {

    return Collections.unmodifiableList(Arrays.asList(elements));

}
```

3.5 Common Traps

array.equals(b) uses Object.equals — it's a reference check. Always use Arrays.equals(a, b) for element-wise comparison.

Arrays.asList(int[]) returns List of size 1, not List. Use Integer[] or stream to get a proper List.

Arrays.asList() returns a fixed-size List backed by the array. .add() throws UnsupportedOperationException. Wrap in new ArrayList<>(...) if you need mutability.

varargs + null: handle(null) passes null as the array itself (NPE). handle((String) null) passes a String[]{null} with one null element.

Array covariance: String[] is Object[], but writing a non-String element throws ArrayStoreException at runtime. Generics are invariant — List is NOT List — safer design.

4. Class vs Object

4.1 Core Distinction

A **class** is a type definition — it exists once in Metaspace, loaded by the ClassLoader, and describes structure and behaviour. A **object** is a runtime instance — allocated on the heap with its own copy of instance fields, linked back to its class via a klass pointer for method resolution.

Dimension	Class	Object
What it is	Type definition / blueprint	Runtime instance
Where it lives	Metaspace	Heap
How many	Once per ClassLoader	As many as new is called
When created	Class loading (first use)	new keyword at runtime
Contains	Field types, method bytecode, static data	Instance field values + klass pointer
Method code	Stored here (once, shared)	Not stored — resolved via klass ptr
Accessed via	Class name: Car.getCount()	Reference variable: bmw.accelerate()
In JVM	java.lang.Class instance in Metaspace	Object instance on Heap

4.2 Object Creation — JVM Steps

When new BankAccount("Alice", 500) executes:

1. Check if BankAccount.class is loaded → if not, invoke ClassLoader
2. Allocate heap memory (object header + instance field slots, zero-filled)
3. Set the klass pointer in the object header → points to BankAccount in Metaspace
4. Execute the constructor (<init> method) to fill in field values
5. Return the reference to the caller

// Bytecode:

new #2 // allocate uninitialised object, push ref

dup // duplicate ref (one for constructor, one to return)

ldc #3 // push "Alice"

ldc2_w #4 // push 500.0

```
invokespecial #5 // call <init> - consumes one ref
```

4.3 The Object Header

Every object on the heap begins with a header you never see but always pay for:

[illegible]

■ Mark word (8 bytes) ■

- → hashCode, GC age, lock state, ■

■ biased-lock thread ID ■

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

■ Klass pointer (4 bytes compressed oops) ■

■ → points to class metadata in Metaspace ■

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90 91 92 93 94 95 96 97 98 99 100 101 102 103 104 105 106 107 108 109 110 111 112 113 114 115 116 117 118 119 120 121 122 123 124 125 126 127 128 129 130 131 132 133 134 135 136 137 138 139 140 141 142 143 144 145 146 147 148 149 150 151 152 153 154 155 156 157 158 159 160 161 162 163 164 165 166 167 168 169 170 171 172 173 174 175 176 177 178 179 180 181 182 183 184 185 186 187 188 189 190 191 192 193 194 195 196 197 198 199 200 201 202 203 204 205 206 207 208 209 210 211 212 213 214 215 216 217 218 219 220 221 222 223 224 225 226 227 228 229 230 231 232 233 234 235 236 237 238 239 240 241 242 243 244 245 246 247 248 249 250 251 252 253 254 255 256 257 258 259 260 261 262 263 264 265 266 267 268 269 270 271 272 273 274 275 276 277 278 279 280 281 282 283 284 285 286 287 288 289 290 291 292 293 294 295 296 297 298 299 300 301 302 303 304 305 306 307 308 309 310 311 312 313 314 315 316 317 318 319 320 321 322 323 324 325 326 327 328 329 330 331 332 333 334 335 336 337 338 339 340 341 342 343 344 345 346 347 348 349 350 351 352 353 354 355 356 357 358 359 360 361 362 363 364 365 366 367 368 369 370 371 372 373 374 375 376 377 378 379 380 381 382 383 384 385 386 387 388 389 390 391 392 393 394 395 396 397 398 399 400 401 402 403 404 405 406 407 408 409 410 411 412 413 414 415 416 417 418 419 420 421 422 423 424 425 426 427 428 429 430 431 432 433 434 435 436 437 438 439 440 441 442 443 444 445 446 447 448 449 450 451 452 453 454 455 456 457 458 459 460 461 462 463 464 465 466 467 468 469 470 471 472 473 474 475 476 477 478 479 480 481 482 483 484 485 486 487 488 489 490 491 492 493 494 495 496 497 498 499 500 501 502 503 504 505 506 507 508 509 510 511 512 513 514 515 516 517 518 519 520 521 522 523 524 525 526 527 528 529 530 531 532 533 534 535 536 537 538 539 540 541 542 543 544 545 546 547 548 549 550 551 552 553 554 555 556 557 558 559 560 561 562 563 564 565 566 567 568 569 570 571 572 573 574 575 576 577 578 579 580 581 582 583 584 585 586 587 588 589 590 591 592 593 594 595 596 597 598 599 600 601 602 603 604 605 606 607 608 609 610 611 612 613 614 615 616 617 618 619 620 621 622 623 624 625 626 627 628 629 630 631 632 633 634 635 636 637 638 639 640 641 642 643 644 645 646 647 648 649 650 651 652 653 654 655 656 657 658 659 660 661 662 663 664 665 666 667 668 669 670 671 672 673 674 675 676 677 678 679 680 681 682 683 684 685 686 687 688 689 690 691 692 693 694 695 696 697 698 699 700 701 702 703 704 705 706 707 708 709 710 711 712 713 714 715 716 717 718 719 720 721 722 723 724 725 726 727 728 729 730 731 732 733 734 735 736 737 738 739 740 741 742 743 744 745 746 747 748 749 750 751 752 753 754 755 756 757 758 759 760 761 762 763 764 765 766 767 768 769 770 771 772 773 774 775 776 777 778 779 780 781 782 783 784 785 786 787 788 789 790 791 792 793 794 795 796 797 798 799 800 801 802 803 804 805 806 807 808 809 810 811 812 813 814 815 816 817 818 819 820 821 822 823 824 825 826 827 828 829 830 831 832 833 834 835 836 837 838 839 840 841 842 843 844 845 846 847 848 849 850 851 852 853 854 855 856 857 858 859 860 861 862 863 864 865 866 867 868 869 870 871 872 873 874 875 876 877 878 879 880 881 882 883 884 885 886 887 888 889 890 891 892 893 894 895 896 897 898 899 900 901 902 903 904 905 906 907 908 909 910 911 912 913 914 915 916 917 918 919 920 921 922 923 924 925 926 927 928 929 930 931 932 933 934 935 936 937 938 939 940 941 942 943 944 945 946 947 948 949 950 951 952 953 954 955 956 957 958 959 960 961 962 963 964 965 966 967 968 969 970 971 972 973 974 975 976 977 978 979 980 981 982 983 984 985 986 987 988 989 990 991 992 993 994 995 996 997 998 999 1000 1001 1002 1003 1004 1005 1006 1007 1008 1009 1010 1011 1012 1013 1014 1015 1016 1017 1018 1019 1020 1021 1022 1023 1024 1025 1026 1027 1028 1029 1030 1031 1032 1033 1034

■ Instance fields (your data) ■

123456789101112131415161718192021222324252627282930313233343536373839404142434445464748495051525354555657585960616263646566676869707172737475767778798081828384858687888990919293949596979899100

A BankAccount with String + double costs:

12 bytes header + 4 bytes padding + 8 bytes double + 4 bytes ref

= ~28 bytes, padded to 32 bytes (JVM aligns to 8-byte boundaries)

4.4 Code Example

```
public class Car {

    // Static field – belongs to CLASS, shared by ALL objects

    private static int totalCarsProduced = 0;

    // Instance fields – each OBJECT gets its own copy

    private String brand;

    private int speed;

    public Car(String brand) {
```

```

this.brand = brand;

this.speed = 0;

totalCarsProduced++; // shared counter across all instances
}

// Instance method – 'this' is the hidden first argument
public void accelerate(int amount) { this.speed += amount; }

// Static method – no 'this', class-level data only
public static int getTotalCarsProduced() { return totalCarsProduced; }
}

// Usage:

Car bmw = new Car("BMW"); // Object 1: brand="BMW", speed=0

Car tesla = new Car("Tesla"); // Object 2: brand="Tesla", speed=0

bmw.accelerate(100); // bmw.speed=100; tesla.speed unchanged

System.out.println(Car.getTotalCarsProduced()); // 2 – class-level counter

// Same class object for all instances:

System.out.println(bmw.getClass() == tesla.getClass()); // true

// Two references, one object:

Car alias = bmw;

alias.accelerate(50);

System.out.println(bmw.getSpeed()); // 150 – same heap object!

System.out.println(bmw == alias); // true – same reference

```

4.5 this — The Hidden Parameter

When you call `bmw.accelerate(100)`, the JVM translates this to `Car.accelerate(bmw, 100)`. `this` is not magic — it is a hidden first argument passed to every instance method. The method code lives in the class (once); `this` tells it which object's fields to read/write. Static methods have no `this` — they operate on

the class, not any instance.

4.6 null — A Reference to No Object

```
Car myCar = null; // stack variable exists, points to nothing

myCar.accelerate(50); // NullPointerException

// JVM tries to read klass pointer from object header

// but there is no object — no header to read
```

4.7 Interview Traps

Static methods cannot access instance fields — no "this" exists in a static context.

The class itself is an object: `Car.class` and `bmw.getClass()` both return the same `java.lang.Class` instance in Metaspace. This is the basis of reflection.

Two references, one object: `Car b = a` does NOT copy the `Car`. Mutating via `b` also changes what `a` sees — they reference the same heap address.

Static fields shared across all instances — not thread-safe by default. Use `AtomicInteger` for shared counters in multithreaded code.

Interview signal: When asked "class vs object", go beyond "blueprint vs instance". Mention Metaspace vs Heap, the klass pointer in the object header, how method code lives in the class but "this" ties execution to a specific instance. That shows JVM understanding, not just syntax knowledge.